

WAFFLESTUDIO, FALL 2026

Product Engineering

System design & setup

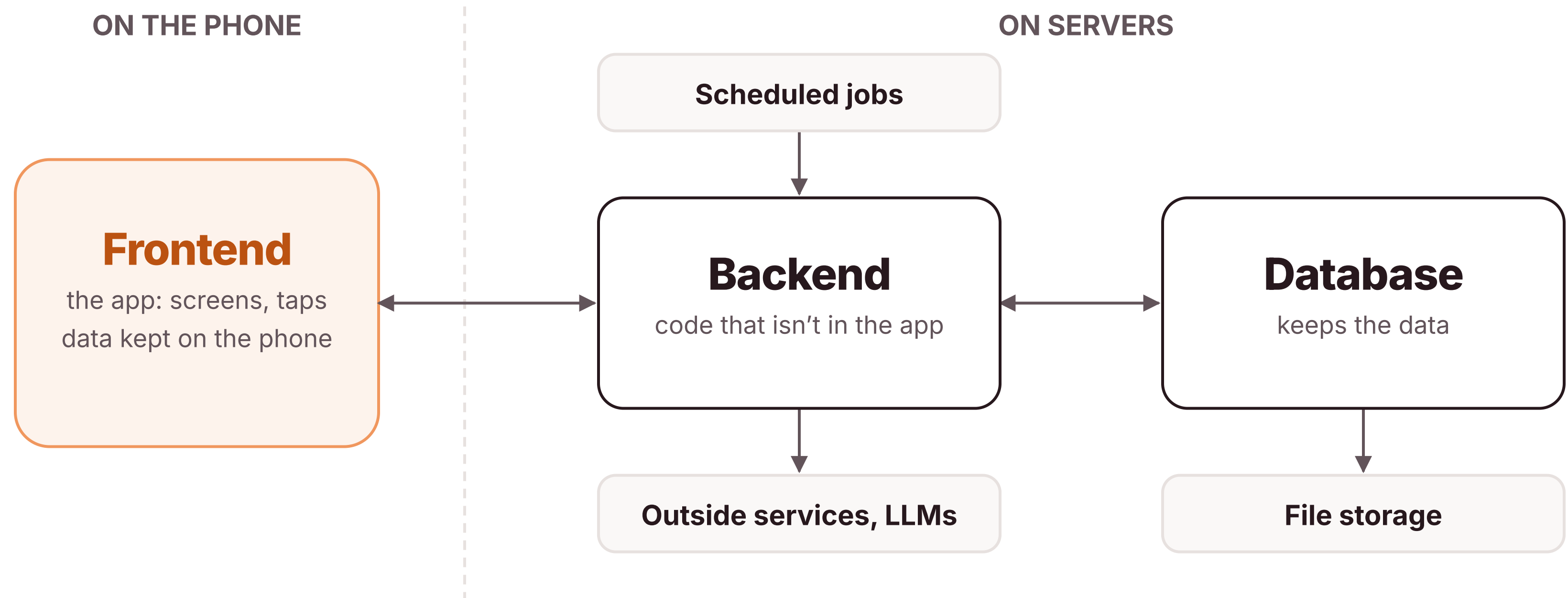
Session 02, 2026/09/29
Chaehyun Park

You drew the screens.
Now, what runs behind them.

01 **The parts of an app**

Frontend, backend, and what sits around them

Frontend and backend



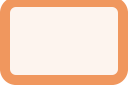
What runs where

	Frontend	Backend
Runs on	the user's phone	servers you control
Who can read it	anyone with the app	only you
Good for	screens, taps, data for one person	shared data, secret keys, scheduled work

Anything secret or shared goes on the backend.

What on the screen is **data**?



-  **Data, from the backend**
Films, posters, showtimes, travel time. Different every day.
-  **The app itself**
Tabs, filters, layout. The same for everyone.

02 **Frontend**

How a screen is made, on the web and on a phone

A web page: **HTML**, CSS, JavaScript

HTML

what's on the page

```
<button id="like">
  ♥ 12
</button>
```

CSS

how it looks

```
#like {
  background: orange;
  border-radius: 999px;
  color: white;
}
```

JavaScript

what it does

```
like.onclick = () => {
  count++;
  like.textContent =
    "♥ " + count;
};
```

The browser shows

♥ 13

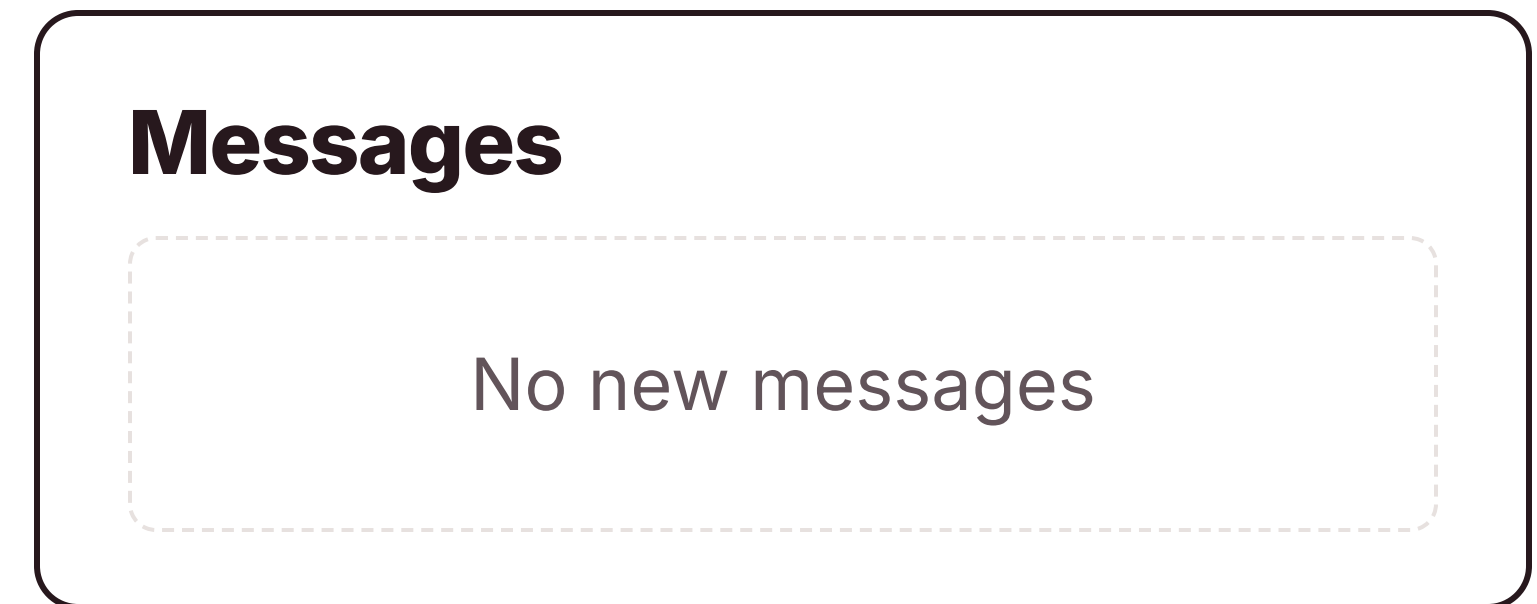
A new message that **wasn't there**

The badge



≠

The inbox



The badge and the inbox each tracked the count. **They stopped agreeing.**

React: change the data once

By hand

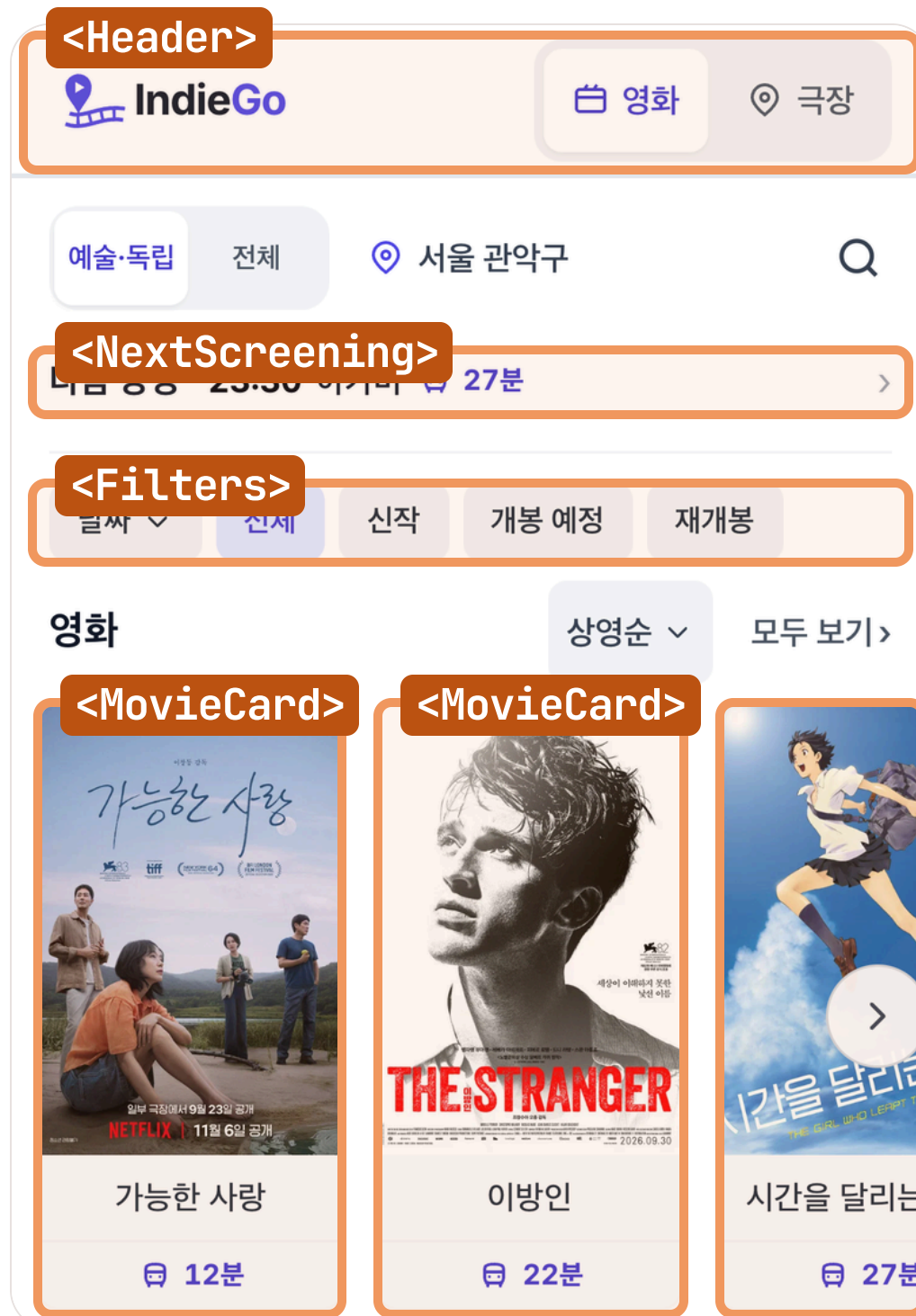
```
function save(film) {  
  saved.push(film);  
  heart.classList.add("on");           // 1  
  badge.textContent = saved.length;    // 2  
  list.append(makeRow(film));          // 3  
}  
  
// unsave() must undo all three.  
// Forget one, and the screen  
// disagrees with itself.
```

React

```
const [saved, setSaved] = useState([]);  
  
function save(film) {  
  setSaved([...saved, film]);  
}  
  
<Heart on={saved.includes(film)} /> // 1  
<Badge count={saved.length} />      // 2  
<SavedList films={saved} />         // 3
```

By hand, every place is your job. **In React, each place reads the data.**

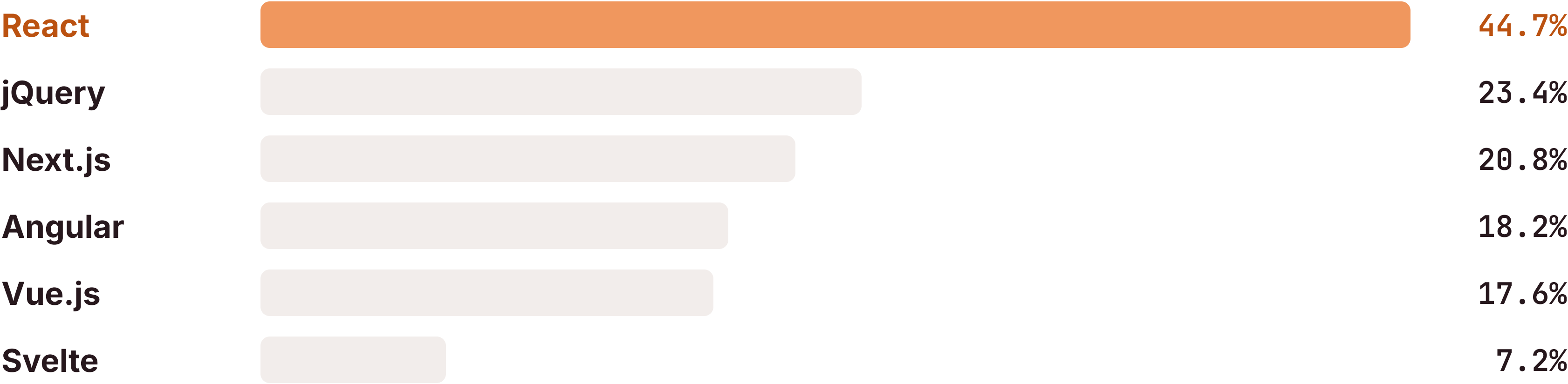
A screen is made of components



```
function Home() {  
  return (  
    <>  
      <Header />  
      <NextScreening />  
      <Filters />  
      {movies.map(m =>  
        <MovieCard movie={m} />  
      )}  
    </>  
  );  
}
```

Write MovieCard once, use it for every film.

What web developers use



Next.js is built on React too.

Three ways to build a phone app

Native

 Swift iOS

 Kotlin Android

Cross-platform

 **React Native** WE USE

 Flutter

A website, in an app

 PWA

 Capacitor

React, for phones. One codebase for iOS and Android.

How they differ

	Native	React Native	Flutter	A website, in an app
Codebases	two	one	one	one
Draws the screen	the OS	the OS	its own engine	a browser view
Language	Swift, Kotlin	TypeScript	Dart	HTML, JavaScript
Phone features	all	through modules	through plugins	limited

React Native: same React, **phone** parts

React, on the web

```
<div>
  <img src={poster} />
  <p>이방인</p>
  <button onClick={book}>
    16:25
  </button>
</div>
```

React Native, on a phone

```
<View>
  <Image source={poster} />
  <Text>이방인</Text>
  <Pressable onPress={book}>
    <Text>16:25</Text>
  </Pressable>
</View>
```

Components, state, useState: **all the same**. Only the tags change.

Each tag is a **box** on the screen



```
<View>
  <Image source={poster} />
  <Text>이방인</Text>
  <Text>아트하우스 모모 · 22분</Text>
  <View style={{ flexDirection: "row" }}>
    <Pressable><Text>16:25</Text></Pressable>
    <Pressable><Text>19:40</Text></Pressable>
  </View>
</View>
```

View a box that holds others

Image a picture

Text any words

Pressable something you tap

Your code becomes **native** views

You write	iOS shows	Android shows	On the web, it's like
<code><View></code>	UIView	ViewGroup	<code><div></code>
<code><Text></code>	UITextView	TextView	<code><p></code>
<code><Image></code>	UIImageView	ImageView	<code></code>
<code><TextInput></code>	UITextField	EditText	<code><input></code>

It looks and feels like any other app, **because it is one.**

React Native, with Expo



Expo Router

A file in app/ becomes a screen.



Native modules

Camera, notifications, location, ready to use.



Expo Go

Run it on your phone. No Xcode needed.



Development build

Your own Expo Go, with any native library.



EAS Build

Builds for the stores, in the cloud.



EAS Submit

Sends it to TestFlight and Google Play.

React Native's own docs recommend starting with a framework, **and name Expo.**

Expo Go, or your own build

Expo Go

Expo's own modules only

"a playground for students and learners"

Development build

Any native library

"your own version of Expo Go"

Needs a development build:

Seeing which app is open

Drawing over other apps

Reading notifications

Widgets

HealthKit, Health Connect

Language: TypeScript

```
type Screening = { cinema: string; startsAt: string; seatsLeft: number }
```

```
showScreening({ cinema: "롯데시네마 신림", startsAt: "16:25" })
```

```
x Property 'seatsLeft' is missing in type '{ cinema: string; startsAt: string; }'
```

Caught before the app runs. **The agent can check its own work** with `npx tsc`.

03

Backend

Where shared data and secret keys live

What a backend is for



Keep shared data

IndieGo: every showtime, the same for every visitor.



Keep secret keys

IndieGo: movie-data API keys never ship to the browser.



Run on a schedule

IndieGo: fetches showtimes six times a day.



Talk to outside services

IndieGo: 15 booking systems, TMDB, KOBIS.

Backend: **Convex**

Your own server

 Spring Boot

 NestJS

 FastAPI

Serverless functions

 AWS Lambda

 Cloudflare Workers

Managed backend

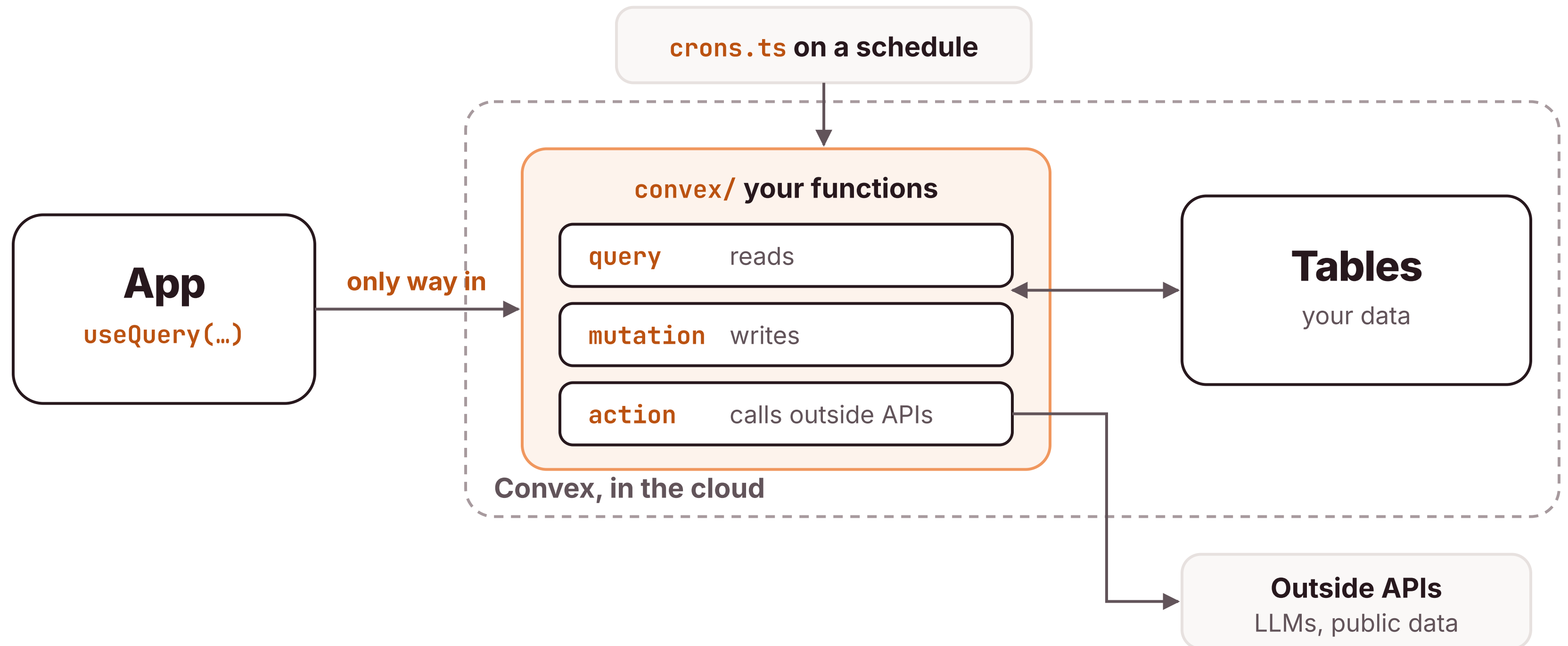
 **Convex** **WE USE**

 Supabase

 Firebase

Database, functions, file storage and scheduled jobs, all in TypeScript.

How Convex works



A query, end to end

convex/screenings.ts

```
export const list = query({
  args: { cinemaId: v.id("cinemas") },
  handler: (ctx, { cinemaId }) =>
    ctx.db.query("screenings")
      .withIndex("by_cinema",
        q => q.eq("cinemaId", cinemaId))
      .collect(),
});
```

In the app

```
const screenings = useQuery(
  api.screenings.list,
  { cinemaId },
);

return screenings?.map(s =>
  <ScreeningCard key={s._id} screening={s} />
);
```

When the data changes, **every screen showing it updates.**

Why Convex, not Supabase

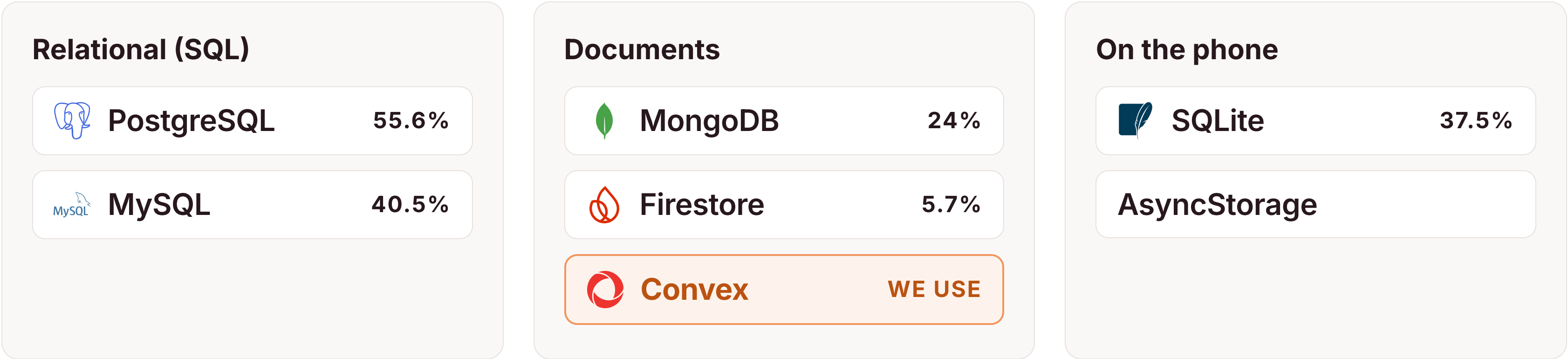
	Convex	Supabase
The app reaches data	through your functions only	straight from the tables
What keeps it safe	checks in your functions	RLS policies, if you turn them on
Language	TypeScript, end to end	SQL, plus Deno for functions
Unused for a week	keeps running	free project pauses
You learn	Convex's way	SQL, used everywhere

04

Data

Tables, and what makes a row unique

Kinds of databases



Tables, or documents?

Tables

cinemas			
c1	롯데시네마 신림		
screenings			
s1	c1	이방인	16:25
s2	c1	가능한 사랑	19:45

Each fact once. Join them when you read.

이방인, tonight? **one query on screenings**

Documents

```
{ name: "롯데시네마 신림",
  screenings: [
    { movie: "이방인", time: "16:25" },
    { movie: "가능한 사랑", time: "19:45" },
  ] }
```

One cinema's page: one read.

이방인, tonight? **open every cinema, look inside**

IndieGo asks by film, **so screenings get their own table.**

Where a document **wins**

curation_programs		
title	ends	description
스탠리 큐브릭 감독전	9/30	{ ... }

- ✓ **Read whole, on one page**
- ✓ **Never searched inside**
- ✓ **Every one a different shape**

```
// description, one JSON document
{ blocks: [
  { type: "heading", level: 2, content: [...] },
  { type: "paragraph", content: [
    { type: "text", text: "9/30까지", strong: true },
  ] },
  { type: "list", ordered: false, items: [...] },
] }
```

Search across it: a table. **Read it whole: a document.**

From the screen to tables

상영일

오늘

9월 13일 (일)

9월 14일 (월)

9월 15일 (화)

9월 16일 (수)

9월 17일 (목)

9월 18일 (금)

이동시간 순

롯데시네마 서울대입구

3분

1관

10:50 - 12:49

120/131

1관

13:10 - 15:09

99/131

3관

15:40 - 17:39

74/78

1관

17:40 - 19:39

89/131

1관

20:00 - 21:59

86/131

1관

22:20 - 24:19

104/131

+1

롯데시네마 신림

13분

4관

09:30 - 11:29

66/84

8관

10:00 - 11:59

72/76

4관

11:50 - 13:49

60/84

4관

16:25 - 18:24

29/84

8관

17:25 - 19:24

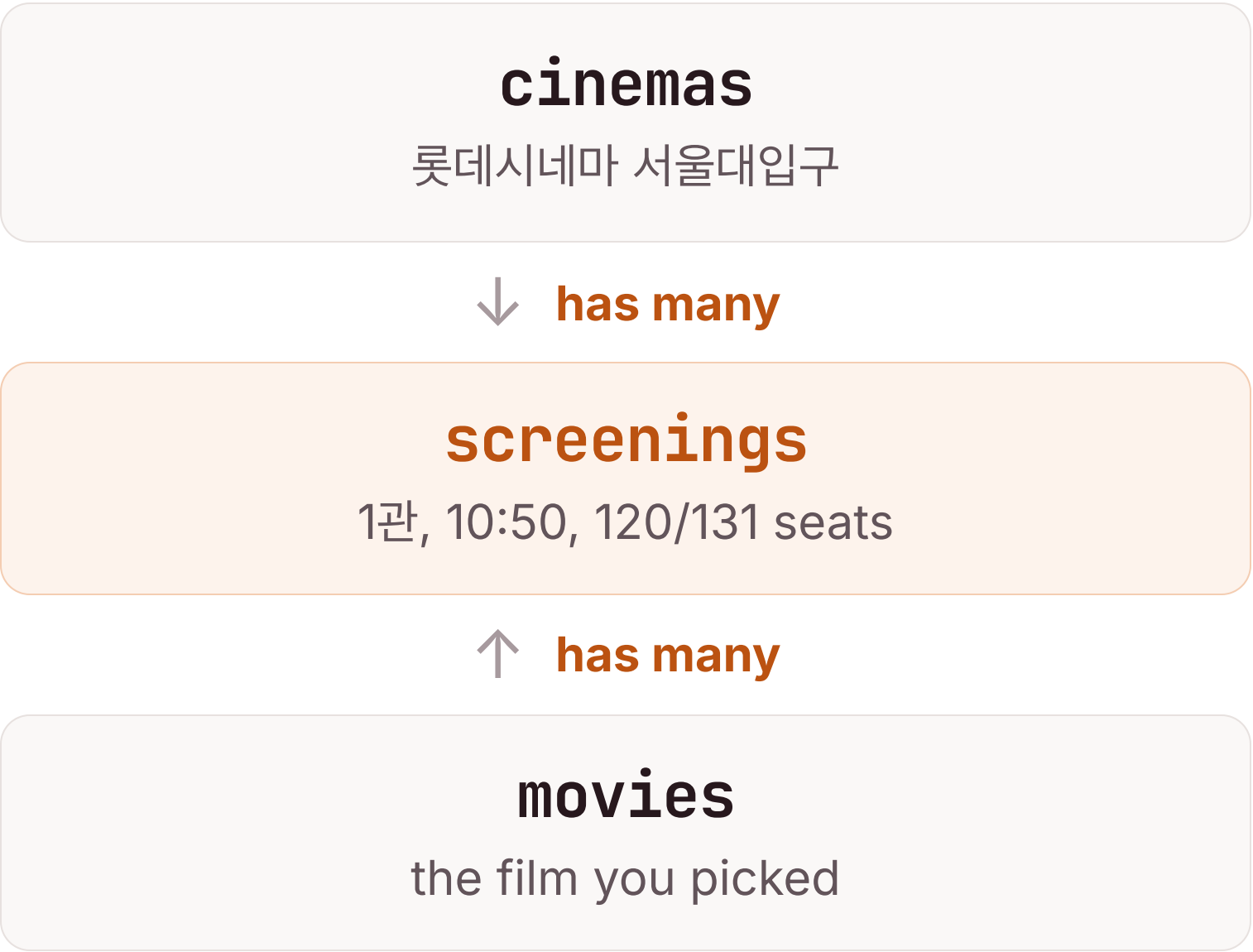
38/76

8관

19:45 - 21:44

16/76

+5



Tables that point to each other

```
// convex/schema.ts
export default defineSchema({
  cinemas: defineTable({ name: v.string() }),
  movies:  defineTable({ title: v.string() }),
  screenings: defineTable({
    cinemaId: v.id("cinemas"),
    movieId:  v.id("movies"),
    startsAt: v.string(),
    screen:   v.string(),
  }).index("by_cinema", ["cinemaId"]),
});
```

A screening stores **the ID of its cinema**, not a copy of the cinema.

What makes two rows the **same**?

From the cinema's site

~~id: LT-88213~~

롯데시네마 신림 · 9/30 19:45 · 4관

From its Instagram

~~id: ig_3f9a~~

신림점 4관, 9월 30일 저녁 7시 45분 GV



THE KEY

cinema + start time + screen

Same key, **one row**. One screen shows one film at a time.

Each source makes up its own ID. **Pick the key before the first row goes in.**

05

The other parts

Keys, safety, outside data, and what to skip

Where the **secret key** goes

In the app

```
EXPO_PUBLIC_LLM_KEY
```

“visible in plain-text in your compiled application”

In a Convex action

```
process.env.LLM_KEY
```

Set with `npx convex env set`. Stays on the server.

Anything in the app can be read by whoever has the app.

Never trust the frontend

170

apps with their database left open

1,645

vibe-coded apps scanned, 2025

Your Convex URL ships inside the app, so anyone can call your functions. **Check inside each one.**

Outside data: **when** do you fetch it?

Once

Rarely changes

A list of places, a public dataset.

`npx convex import`

On a schedule

Changes every day

IndieGo: showtimes, six times a day.

`crons.ts`

When the user asks

Different every time

IndieGo: place search, as you type.

`an action`

Ahead of time when you can. **Live when it depends on the user.**

INDIEGO

What the sources allowed

Source	What happened	Fix
CGV	403 for every request from AWS	go through a proxy
Dtryx	drops connections from data-center IPs	go through a proxy

Call each source once before you design around it.

Add these only if you need them

	Skip it when	Add it when
Login	data stays on one phone	data follows a person, or others see it
Push notifications	the phone knows when (local)	the server decides when
File storage	no photos or files	users upload them

Getting it onto phones: 11/14



Google Play asks new accounts for **12 testers for 14 days** before a public launch.

06

Making architecture decisions

Start from a common shape, then add only what you need

INDIEGO

Start from the screens

Screen	Shows	Comes from
Home	films near you, travel time	fetches showtimes, precomputed times
Film	cinemas and showtimes	fetches showtimes
Cinemas	cinemas on a map	cinema list, Naver Maps

Each “comes from” becomes a part in your diagram.

Four common shapes

A On the phone only



A diary, a timer, a habit tracker

B App + backend



Notes on your phone and laptop

C App + a function that holds a key



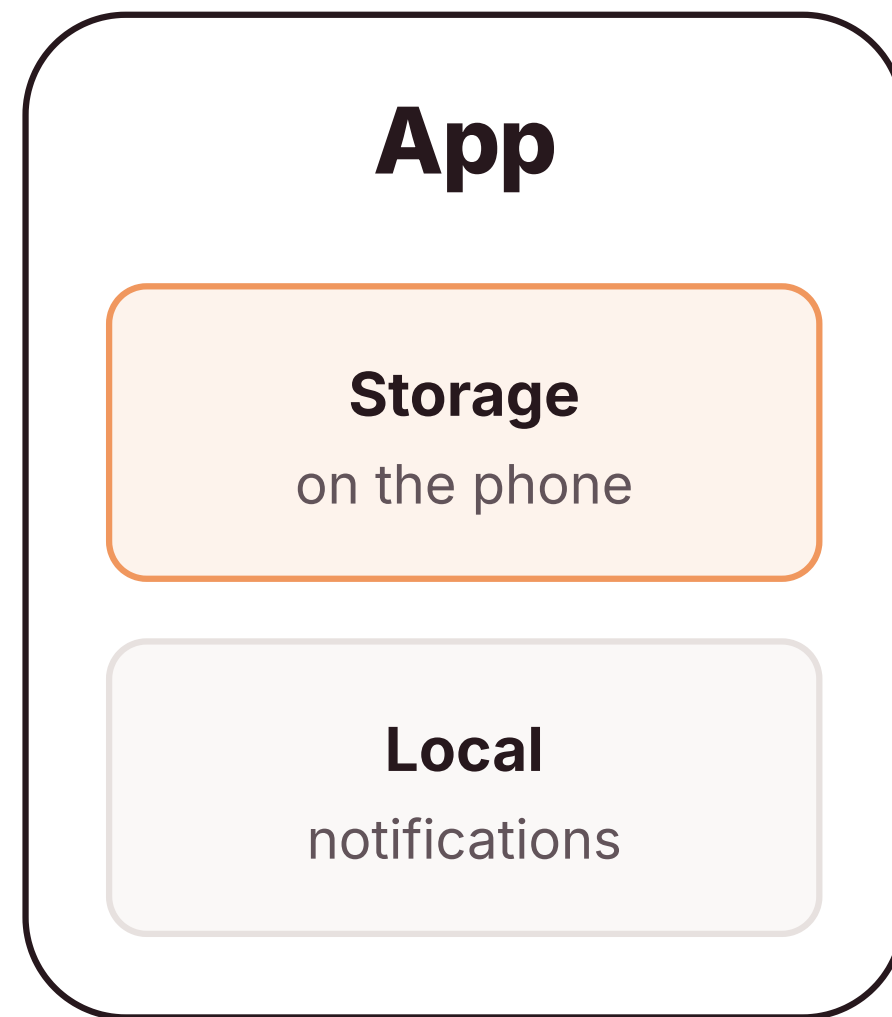
Anything that asks an LLM

D Data collected on a schedule



Showtimes, prices, timetables

A. On the phone only



USE IT WHEN

Everything stays with one person, on one phone.

EXAMPLES

A diary, a timer, a habit tracker

COST

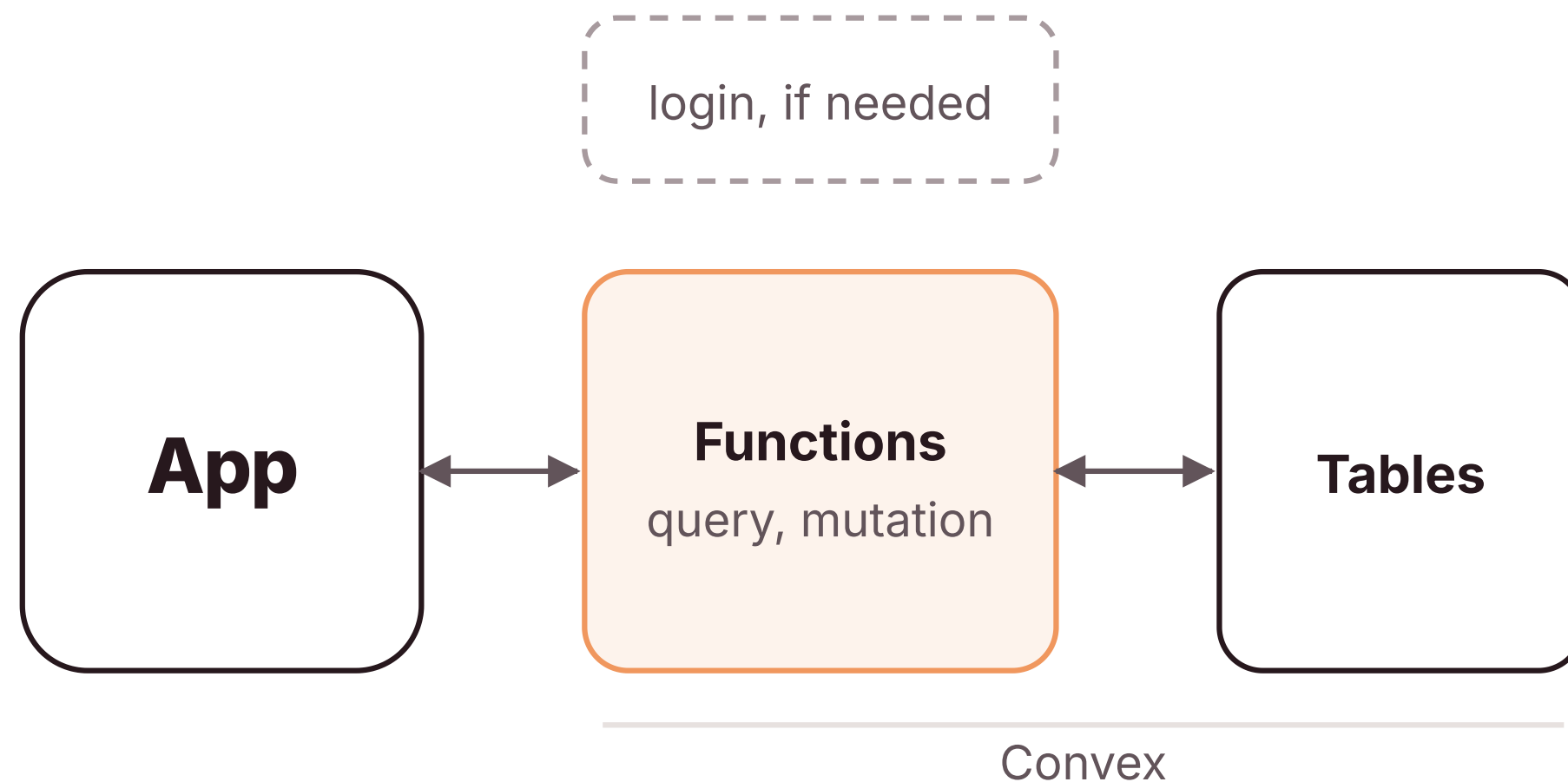
Lose the phone, lose the data.

Storage on the phone

	What it is	Use it for
AsyncStorage	a key and a text value	settings, a draft, "seen the intro"
SQLite	a SQL database in one file	records you search and sort: a diary, expenses
SecureStore	a key and an encrypted value	login tokens, anything private

Shape A: **no server**, and the data still survives a restart.

B. App + backend



USE IT WHEN

Data follows the person to another device, or other people see it.

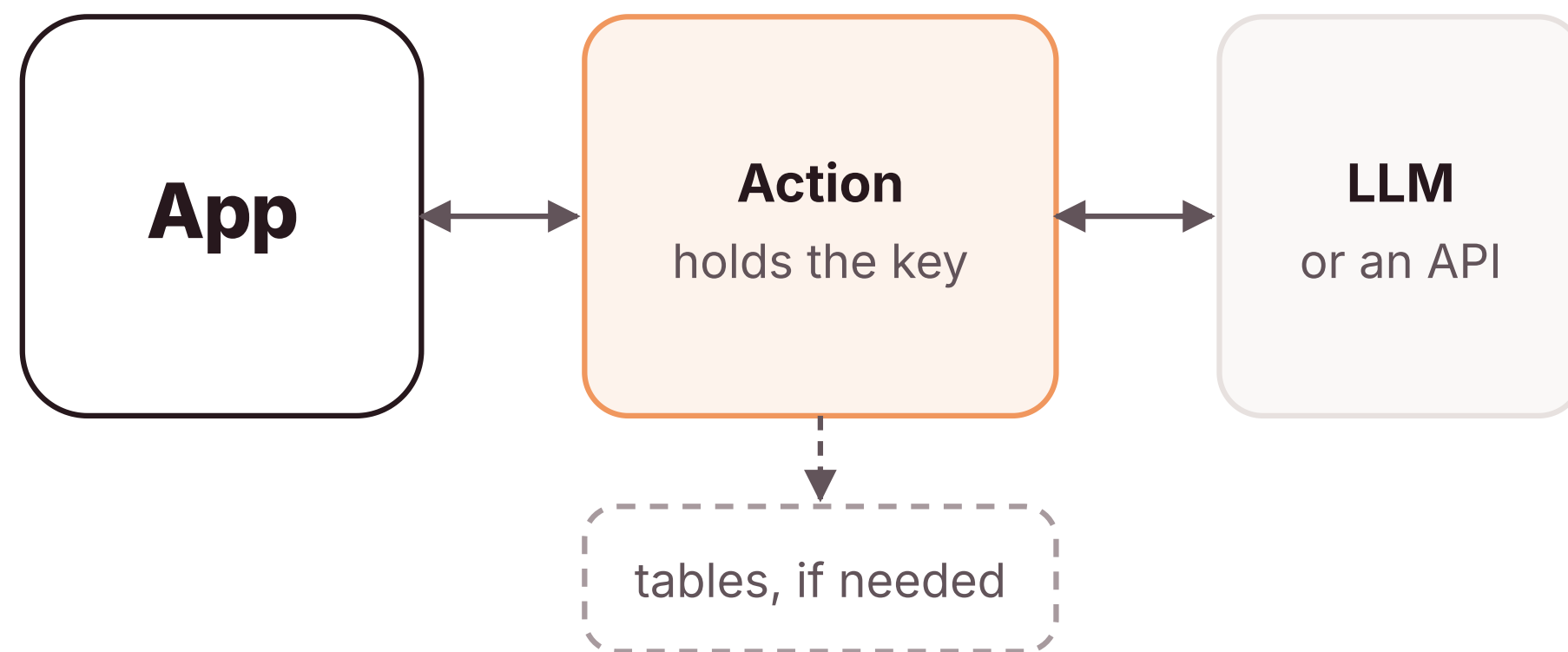
EXAMPLES

Notes on your phone and laptop, a list shared with friends

COST

One more thing to run. You protect the data you keep.

C. App + a function that holds a key



USE IT WHEN

The app calls an LLM, or an API that needs a secret key.

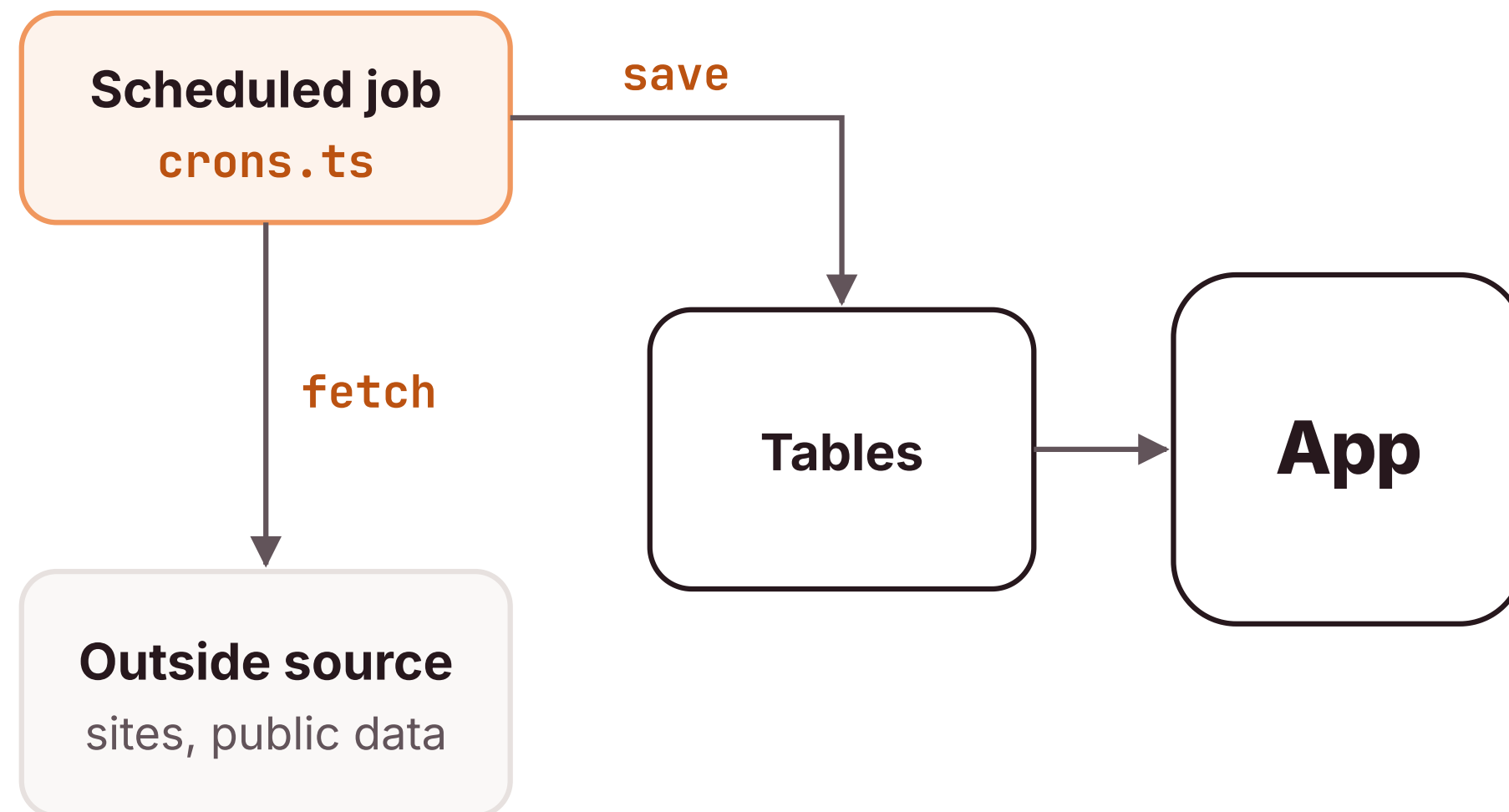
EXAMPLES

Ask in plain words, get an answer;
search a public dataset

COST

Every call takes time and money, and APIs have quotas.

D. Data collected on a schedule



USE IT WHEN

The data changes somewhere else, without your user doing anything.

EXAMPLES

Showtimes, prices, timetables. IndieGo is this shape.

COST

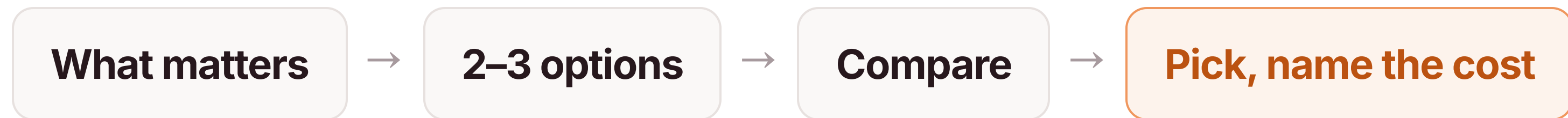
Sources change or block you, and someone has to notice.

Pick the **smallest** shape

Ask	If yes, add
Does any data need to leave the phone?	B a backend
Do you need a secret key? (LLM, paid API)	C an action
Does data change without your user?	D a scheduled job

Start from A. Add only what a yes asks for.

Weigh the options for **your** case



Writing down the options you didn't pick "shows very explicitly why the selected solution is the best **given the project goals.**"

On the phone, or on the backend?

	On the phone	On the backend
Without internet	works	doesn't
Lost phone	data gone	data safe
Other devices, other people	no	yes
Private data	stays with the user	you have to protect it

Where does travel time come from?

What matters	A routing API, per card	Straight-line distance	Computed ahead, looked up
Real transit time	yes	no	yes
A time on every card	quota, cost, slow	yes	yes
Effort to build	low	lowest	high

Every film card shows a travel time, so a call per card doesn't fit.

One-way and **two-way** doors

One-way

Hard to undo. Take your time.

Your tables

What makes a row unique

Phone or backend

Two-way

Easy to undo. Just pick.

Colors and layout

A UI library

Which screen comes first

"most decisions aren't like that – they are changeable, reversible – **they're two-way doors.**"

Choose **boring** technology

Boring, already decided

^ Expo

TS TypeScript

Convex

GitHub

Spend one token

HealthKit

an LLM

a widget

"Let's say every company gets about **three innovation tokens.**"

Write the decision down

The site reads published files, not the database

Context Visitors read tables directly with the public key.

Decision Every 10 minutes, a job publishes the data as files.
The site reads only those files.

Consequences No visitor can reach the database.
Data can be up to 10 minutes old.

Context, decision, consequences. **Add the options you didn't pick, and why.**

07 **Assignment 2**

System design & setup · due 10/30, 23:59

1. The design

What goes in the design

docs/02-system-design.md

Overview

your system in a few lines: the parts, and why each is there

Diagram

the parts you use, with the ones you don't crossed out

Core flow

your main flow, step by step: which part does each step

Data

your tables, and what makes each row unique

Data sources

how you get each one, what a real call returned, its limits

Decisions

two or three: options, what you picked, what it costs

Under 800 words, plus the diagram.

2. The setup

Set up the project

- 1 **Create the Expo app** `npx create-expo-app@latest mobile`
- 2 **Run it on your phone**
Expo Go, or a development build if you need one.
- 3 **Add Convex** if your system has a backend `npx convex dev`
- 4 **Turn on CI**
A type check on every PR.

Step by step, with the CI file: [handouts/02-system-design.md](#)

3. The proof

Prove the **riskiest** part

Call it for real



Save what came back



Link it in the doc

Your hardest data source, from a Convex action or a small script. **Put the response in docs/samples/.**

Needs a phone permission instead? Show it working in a development build.

What to hand in

- 1 The design** docs/02-system-design.md
Diagram as docs/architecture.png, embedded in the doc.
- 2 The app**, in the same repo
Runs on your phone, CI green.
- 3 The proof** docs/samples/
A real response, or a recording of the permission working.

Due 10/30, 23:59. Same repo, new branch, one PR. Feedback comes back as review comments.

Classes and due dates

9/12	01	Problem
9/22		Pitch & sketch due 23:59
9/29	02	Design & setup remote, today
		midterms, no class for five weeks
10/30		Design & setup due 23:59
10/31	03	Build end to end
11/6		End to end due 23:59
11/7	04	MVP
11/13		MVP due 23:59
11/14	05	Ship
11/20		Release due 23:59
11/21	06	Demo day

Reading

Software Engineering for Vibe Coders, technically.dev

Frontends + Backends · Databases + Storage
Auth + Security

Convex tutorial, Convex

A working app with a backend, in an hour

Choose Boring Technology, Dan McKinley

Innovation tokens

Design Docs at Google, Malte Ubl

Trade-offs, alternatives considered

References

technically.dev — On Frontends + Backends

technically.dev/posts/software-eng-vibe-coders-frontends-backends

technically.dev — Auth + Security

technically.dev/posts/security-for-vibe-coders

MDN — What is JavaScript?

developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/What_is_JavaScript

React — Describing the UI

react.dev/learn/describing-the-ui

Stack Overflow Developer Survey 2025

survey.stackoverflow.co/2025/technology

React Native — Core Components and Native Components

reactnative.dev/docs/intro-react-native-components

React Native — Get Started

reactnative.dev/docs/environment-setup

Expo — FAQ

docs.expo.dev/faq

Expo — Environment variables

docs.expo.dev/guides/environment-variables

Convex — Functions

docs.convex.dev/functions

Convex — Schemas

docs.convex.dev/database/schemas

Convex — React Native quickstart

docs.convex.dev/quickstart/react-native

Convex — Cron jobs

docs.convex.dev/scheduling/cron-jobs

Jeff Bezos — 2015 Letter to Shareholders

sec.gov/Archives/edgar/data/1018724

Dan McKinley — Choose Boring Technology

mcfunley.com/choose-boring-technology

Michael Nygard — Documenting Architecture Decisions

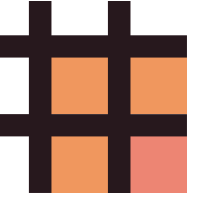
cognitect.com/blog/2011/11/15/documenting-architecture-decisions

Malte Ubl — Design Docs at Google

industrialempathy.com/posts/design-docs-at-google

MADR — Markdown Architectural Decision Records

adr.github.io/madr



NEXT

03

Build end to end

10/31 13:00